ACTIVIDAD PW2 2026 – MICROSERVICIOS CON YII2

Objetivo

Desarrollar una arquitectura basada en microservicios usando PHP + Yii2 + API REST, aplicando CRUD, paginación, búsqueda e integración entre servicios.

CONTEXTO

Cada estudiante desarrollará un microservicio independiente asociado a una tabla de la base de datos Northwind/W3Schools.

Cada microservicio deberá:

```
- Tener su propia base de datos
- Implementar CRUD completo
- Implementar API REST
- Implementar paginador
- Implementar buscador
- Exponer endpoints JSON
- Integrarse con otros microservicios mediante API
```

ORGANIZACIÓN

Cada grupo tendrá 8 integrantes

Cada integrante desarrollará 1 microservicio

Ejemplo:

```
Integrante 1 → Categories
Integrante 2 → Products
Integrante 3 → Customers
Integrante 4 → Orders
Integrante 5 → Employees
Integrante 6 → Suppliers
Integrante 7 → Shippers
Integrante 8 → OrderDetails
```

REGLAS IMPORTANTES

```
NO se permite acceso directo a la base de datos de otro compañero.
Toda comunicación debe realizarse mediante API REST.
```

Ejemplo correcto:

```
Products consume Categories API
```

Ejemplo incorrecto:

```
Products conectándose directamente a la BD Categories
```

REQUERIMIENTOS TÉCNICOS

Tecnologías

```
- PHP 8+
- Yii2
- MySQL
- XAMPP
- API REST
- JSON
```

FUNCIONALIDADES OBLIGATORIAS

CRUD COMPLETO

```
- Listar
- Registrar
- Editar
- Eliminar
- Ver detalle
```

PAGINADOR

Mínimo:

```
10 registros por página
```

BUSCADOR

Debe permitir búsqueda por:

```
- ID
- Nombre
- Texto relacionado
```

API REST OBLIGATORIA

Ejemplo:

```
GET    /products
GET   /products/1
POST   /products
PUT   /products/1
```

```
DELETE /products/1
```

INTEGRACIÓN ENTRE MICROSERVICIOS

Ejemplo:

```
Products debe consumir Categories API
Orders debe consumir Customers API
OrderDetails debe consumir Products API
```

ENTREGABLES

1. Código fuente

```
Proyecto Yii2 completo
```

2. Base de datos

```
Archivo SQL
```

3. Documentación

Debe incluir:

```
- Arquitectura
- Endpoints
- Capturas
- Explicación del flujo
- Integraciones API
```

4. Exposición

Cada estudiante deberá explicar:

```
- Arquitectura del microservicio
- CRUD
- API REST
- Consumo de APIs
- Paginador
- Buscador
```

CRITERIOS DE EVALUACIÓN

```
20% Arquitectura
20% CRUD funcional
20% API REST
15% Integración entre microservicios
15% Paginador y buscador
10% Exposición técnica
```

RECOMENDACIÓN

```
Primero desarrollar CRUD local.
Luego convertirlo en API REST.
Finalmente integrar microservicios.
```

1. Microservicio Categorías

Tabla:

```
CREATE TABLE categories (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL
);
```

Modelo `models/Category.php`:

```
class Category extends \yii\db\ActiveRecord {
      public static function tableName() {
      return 'categories';
      }
}
```

Controlador `controllers/CategoryController.php`:

```
namespace app\controllers;

use yii\rest\ActiveController;

class CategoryController extends ActiveController {
      public $modelClass = 'app\models\Category';
}
```

En `config/web.php`:

```
'urlManager' => [
      'enablePrettyUrl' => true,
      'showScriptName' => false,
      'rules' => [
      ['class' => 'yii\rest\UrlRule', 'controller' => 'category'],
      ],
],
```

API:

```
GET   /categories
POST   /categories
GET   /categories/1
PUT    /categories/1
DELETE /categories/1
```

---

2. Microservicio Productos

Tabla:

```
CREATE TABLE products (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  price DECIMAL(10,2) NOT NULL,
  category_id INT NOT NULL
);
```

Modelo `models/Product.php`:

```
class Product extends \yii\db\ActiveRecord {
      public static function tableName() {
      return 'products';
      }
}
```

Controlador `controllers/ProductController.php`:

```
namespace app\controllers;

use yii\rest\ActiveController;
use yii\helpers\Json;

class ProductController extends ActiveController {
      public $modelClass = 'app\models\Product';

      public function actionWithCategory() {
      $products = \app\models\Product::find()->asArray()->all();

      foreach ($products as &$product) {
            $url = 'http://192.168.1.10:8081/categories/' .
$product['category_id'];
            $categoryJson = @file_get_contents($url);
            $category = Json::decode($categoryJson);

            $product['category_name'] = $category['name'] ?? 'Sin
categoría';
      }

      return $products;
      }
}
```

En `config/web.php`:

```
'rules' => [
      ['class' => 'yii\rest\UrlRule', 'controller' => 'product'],
      'products-with-category' => 'product/with-category',
],
```

API:

```
GET /products
GET /products-with-category
```

Resultado esperado:

```
[
```

```
  {
    "id": 1,
    "name": "Laptop",
      "price": "2500.00",
      "category_id": 2,
      "category_name": "Tecnología"
  }
]
```


La regla es: **Productos no tiene tabla categories; solo consume la API del microservicio Categorías.**